DSA UNIT 2 - MCQ PRACTICE

Questions Only - Answers at the End

NOTE: The last part, i.e. the last 30 questions, has not yet been uploaded. Hence, those 30 questions are not included in this PDF. This document contains only the questions that were present in the three uploaded files.

Uploaded coverage: Doubly/Circular Linked Lists, Queue, and Priority Queue.

Part A - Doubly & Circular Linked Lists (20 questions)

1) Which operation is more efficient in a doubly linked list compared to a singly linked list?

A. Traversal from head to tail
B. Insertion at the end
C. Deletion of a given node
D. Searching for a key

2) head is a pointer to one of the nodes in the circular list.

```
p = head;
while (p != head) {
   printf("%d ", p->data);
   p = p->next;
}
```

If head != NULL, how many nodes are printed?

A. 0
B. 1
C. Number of nodes
D. Infinite

3) Which condition can be used to detect completion of traversal in a circular linked list?

A. ptr == NULL
B. ptr == head after at least one traversal
C. ptr->next == NULL
D. ptr->prev == NULL

4) Consider:

```
p = head;
while (p != NULL) {
   printf("%d ", p->data);
   p = p->next;
}
```

This code is used to traverse a circular linked list where head points to one of the nodes in the circular list. What happens?

A. All nodes are printed exactly once
B. Only the first node is printed
C. The loop becomes infinite
D. Segmentation fault occurs immediately

5) p points to a node in DLL. If p->prev_ is NULL, it implies

A. list has only one node

6) struct dll { node_t* head_; node_t* tail_; };
typedef struct dll dll_t;
There is no header or tailer node in the list. How to check whether the list is empty?

A. both head_ and tail_ should be NULL.
B. head_ should be NULL and tail_ may not be NULL
C. tail_ should be NULL and head_ may not be
D. none of these

7)

```
void traverse(clist_t* cl) {
   node_t* temp = cl->head;
   do {
      printf("%d ", temp->key);
      temp = temp->next;
   } while (temp != cl->head);
}
```

What will happen if cl->head is NULL?

A. Prints nothing and exits safely
B. Infinite loop
C. undefined behaviour
D. Prints garbage values

8) struct node { int key_; struct node *next_; };
typedef struct node node_t;

struct clist { node_t* head_; };
typedef struct clist clist_t;

```
void add(clist_t *ptr_clist, int val) {
   node_t* temp = create_node(val);
   if (ptr_clist->head_ == NULL) {
      ptr_clist->head_ = temp;
      temp->next_ = temp;
   } else {
      temp->next_ = ptr_clist->head_->next_;
      ptr_clist->head_->next_ = temp;
   }
   // new code
   ptr_clist->head_ = ptr_clist->head_->next_;
}
```

What is the effect of new code added?

A. insertion happens before the node to which head_ points to
B. insertion happens after the node to which head_ points to
C. function will crash while adding to an empty circular list
D. the node to which head_ points to gets deleted

9) struct node { int key_; struct node* prev_; struct node* next_; };
typedef struct node node_t;
temp points a node in the middle of a doubly linked list.

```
temp->next_->prev_ = temp;
```

What is the effect of this code?

A. node to which temp points to gets deleted
B. node next to the one to which temp points to gets deleted
C. node prev to the one to which temp points to gets deleted
D. does not change anything in the list

10) Which statement is true for a circular singly linked list?

A. The last node points to NULL
B. The last node points back to the first node
C. Every node has two links

11) Which application commonly uses circular linked lists?

A. Round-robin scheduling
B. Binary search
C. Recursion
D. Binary tree traversal

12) A circular linked list has exactly one node. If head points to this node, which condition is true?

A. head == NULL
B. head->next == NULL
C. head->next == head
D. head->next->next == NULL

13) A circular linked list is: 5 -> 10 -> 15 -> 20 -> 35 -> back_to_first_node
If p initially points to node 20, what is the value of p->next->next->data?

A. 5
B. 10
C. 15
D. 20

14) Comment.
while (temp->next != NULL) temp = temp->next;
temp->prev->next = NULL;
free(temp);

A. always removes the last node correctly in the DLL
B. always removes the first node correctly in the DLL
C. would have undefined behaviour if the DLL has one node
D. Would result in memory leak

15) A doubly linked list contains:
5 <-> 10 <-> 15 <-> 20
A new node containing 12 is inserted before the node containing 15. Which of the following correctly updates the pointers? p points to the node containing 15.

A. new->next = p; new->prev = p->prev; p->prev = new;
B. new->next = p; new->prev = p->prev; p->prev->next = new; p->prev = new;
C. new->prev = p; new->next = p->next; p->next = new;

16) A pointer called current points to a node in a non-empty circular list.

```
node_t* p = current;
node_t* q = p;
do {
   p = p->next_;
   q = q->next_->next_;
} while (q != current);
```

What would this code do?

A. p would point to a node half a way across from current in the circular list
B. q would point to a node half a way across from current in the circular list
C. both p and q will be same as current when the loop is exited
D. the nodes may be traversed twice if the number of nodes is odd

17) struct dll { node_t* head_; node_t* tail_; };
typedef struct dll dll_t;
There is no header or tailer node in the list.

head_ == tail_ implies

A. either list is empty or list has one node
B. that list is definitely empty
C. that list has only one node
D. none of these

18) A doubly linked list contains n nodes. A pointer p points directly to the node in the middle of the list containing value X. What operation can be performed without a loop?

A. deleting this node requires a loop
B. adding this value to the data in the next node requires a loop
C. inserting a node before p requires a loop
D. adding this value to every node in the list requires a loop

19) Consider the following doubly linked list:
10 <-> 20 <-> 30 <-> 40 <-> 50

A pointer p initially points to the node containing 30. The following operations are performed:

```
p = p->next;
p->prev->next = p->next;
p->next->prev = p->prev;
```

What will be the resulting doubly linked list?

A. 10 <-> 20 <-> 30 <-> 40 <-> 50
B. 10 <-> 20 <-> 40 <-> 50
C. 10 <-> 20 <-> 30 <-> 50
D. 10 <-> 30 <-> 40 <-> 50

20) When we delete an element from a doubly linked list

A. head and tail will never change
B. head could change, tail will never change
C. tail could change, head will never change
D. none of these

Part B - Queue (25 questions)

21) A queue is implemented using a circular list using an array of size 5. The queue uses the convention: front points to the first element; rear points to the position where the next element will be inserted. The queue is empty when front == rear. The queue is full when (rear + 1) % 5 == front. Initially, front = rear = 0. The following operations are performed: ENQUEUE(10), ENQUEUE(20), ENQUEUE(30), DEQUEUE(), ENQUEUE(40), ENQUEUE(50). Which of the following is TRUE?

A. The queue contains 10, 20, 30, 40, 50
B. The queue contains 20, 30, 40, 50 and the queue is full
C. The queue contains 20, 30, 40 and one insertion is still possible
D. The queue becomes full immediately after ENQUEUE(40)

22) dequeue operation

A. always changes the rear
B. might change the rear
C. never changes the rear
D. none of these

23) A queue implemented using a circular list in an array of size 5 uses:
int Q[5]; int front = 0, rear = 0;
with the convention that the queue is empty when front == rear, and one slot is always kept empty. What is the maximum number of elements that can be stored?

A. 5
B. 4
C. 3
D. Depends on the initial value of front

24) In a simple array-based queue without circular reuse, the queue has: front = 2, rear = 5 and the array size is 6. After two dequeues, which problem can occur even though unused locations exist at the beginning of the array?

A. next element will be enqueued at index 0
B. on next enqueue, queue will signal full
C. on next dequeue, queue will signal empty
D. next enqueue will depend on the value of front

25) If both the head (front) and the tail (rear) pointers of a linked-list queue are equal, what does this mathematically imply about the state of the queue?

A. The queue is definitely empty.
B. The queue definitely has exactly one node.
C. It implies either an empty queue or a one-node queue.
D. The queue is full.

26) Consider the following condition in a queue implemented using a circular list:

```
temp = (ptr_queue->rear_ + 1) % MAXSIZE;
ptr_queue->rear_ = temp;
if (!is_full(ptr_queue)) ptr_queue->key_[temp] = key;
```

The modulo operation is primarily used to:

A. Detect an empty queue
B. Detect a full queue
C. Wrap the rear index around the array
D. Shift all queue elements

27) Consider a queue implemented using circular list in an array of size 5. Initially: front = 0, rear = 4. One element is deleted from the queue. Which statement is correct?

A. Queue remains full
B. Queue becomes empty
C. The position at index 0 can be reused
D. No further insertion is possible

28) In the array-based implementation of a queue, what could the front_ and rear_ indices be set to upon initialization?

A. Always -1
B. Always 0
C. Any valid index in the array
D. MAXSIZE

29)

```
void dequeue() {
   Node *p = front;
   front = front->next;
   free(p);
}
```

What problem occurs when the queue contains exactly one element?

A. front becomes NULL but rear remains invalid
B. when front becomes NULL, rear automatically becomes NULL.
C. free(p) is not right; it should be free(Front);
D. front->next is dereferencing a dangling pointer

30) Consider a circular queue with:
#define MAXSIZE 5
front_ = 3; rear_ = 4;
The next enqueue operation computes: temp = (++rear_) % MAXSIZE; What is the value of temp?

A. 5
B. 4
C. 0
D. 1

31) Consider a linear queue implemented using an array of size 5 - not a circular list implementation, with: front = 0, rear = 4. The queue contains five elements. Now, two Dequeue() operations are performed. What happens when an attempt is made to perform Enqueue(60) using the standard linear-queue condition:
if (rear == MAX - 1) printf("Overflow");

A. 60 is inserted at index 0
B. 60 is inserted at index 2
C. Overflow occurs even though there are two unused positions
D. The queue automatically shifts all elements to the left

32) A printer receives print jobs in the following order: Job A -> Job B -> Job C -> Job D. Each job is processed completely before the next job is selected. Which data structure is suitable for this task?

A. queue
B. priority queue
C. stack
D. circular list with time slicing

33) A queue is used to manage customers at a single-counter bank, strictly following FIFO. A new policy allows a manager to pull out and serve any customer from the middle of the line immediately, then the line continues as before. What does this scenario violate?

A. The rule that a queue cannot be implemented using arrays
B. The requirement that a queue must have a fixed capacity
C. The fundamental FIFO property that defines a queue
D. The rule that a queue must always be empty before insertion

34) Assume that both front and rear refer to filled positions in the array implementation of the queue. Consider an array implementation of a queue with front = 2 and rear = 5 and array size is 10. How many elements are currently present?

A. 2
B. 3
C. 4
D. 5

35) A linear queue is implemented using an array Q[0...4]. Initially, front = rear = -1. The following operations are performed:
Enqueue(10); Enqueue(20); Enqueue(30); Dequeue(); Enqueue(40); Enqueue(50);

Which of the following is the correct final state?

A. Q = [20, 30, 40, 50, _], front = 1, rear = 4
B. Q = [_, 20, 30, 40, 50], front = 1, rear = 4
C. Q = [20, 30, 40, 50, _], front = 0, rear = 3
D. Overflow occurs during Enqueue(50)

36) In a queue implemented using a circular list in an array of size N, the next position of rear is calculated as:

A. rear + 1
B. rear - 1
C. (rear + 1) % N
D. (rear - 1) % N

37) Consider the following operations on an initially empty queue:
enqueue(10)
enqueue(20)
dequeue()
enqueue(30)
dequeue()

What is the final element in the queue?

A. 10
B. 20
C. 30
D. Queue is empty

38) A queue is implemented using two stacks, S1 and S2. Enqueue always pushes onto S1. Dequeue first checks if S2 is empty - if so, it pops all elements from S1 and pushes them onto S2, then pops from S2. If S2 is not empty, it pops directly from S2. Given the operations: Enqueue(1), Enqueue(2), Dequeue(), Enqueue(3), Dequeue(), Dequeue(), Enqueue(4), Dequeue() - what is the sequence of values returned by the four Dequeue() calls, in order?

A. 1, 2, 3, 4
B. 4, 3, 2, 1
C. 1, 3, 2, 4
D. 2, 1, 4, 3

39) A queue performs: enqueue(10), enqueue(20), enqueue(30), dequeue(), dequeue(). What is now at the front of the queue?

A. 10
B. 20
C. 30
D. Queue is empty

40) Queue Using a Linked List During an enqueue or dequeue operation on a linked-list queue, under what specific conditions do both the front and rear pointers need to be modified?

A. During every enqueue and dequeue operation.
B. Only when the queue reaches its maximum capacity constraint.
C. When enqueueing the very first node (modify front), or when dequeueing the only remaining node (modify rear) in the queue.
D. When calling free() on the front of a non-empty queue with multiple nodes.

41) A queue is implemented using a singly linked list with pointers front and rear. Which operations can both be performed in time independent of the number of elements in the queue?

A. enqueue only
B. dequeue only
C. enqueue and dequeue
D. Neither

42) In a linked queue, when the last remaining element is deleted, how should the fields be reset.

A. front = rear = NULL
B. front = rear = 0
C. rear = NULL and no change to front
D. front = NULL and no change to rear

43) In a list implementation of a queue,

A. it is possible that the front is NULL and the rear is not
B. it is possible that the rear is NULL and the front is not
C. rear = front definitely implies that the queue is empty
D. none of these

44) Consider: pq->rear++; pq->data[pq->rear] = key; This code is most likely part of:

A. Dequeue
B. Enqueue
C. Peek
D. Initialization

45) A printer receives print jobs in the following order: Job A -> Job B -> Job C -> Job D. The jobs are processed in the order in which they arrive. Which is the best data structure to hold these jobs?

A. queue
B. stack
C. priority queue
D. circular list with time slicing

Part C - Priority Queue (25 questions)

46) A hospital emergency department uses a priority queue to manage patients. A smaller priority number indicates a more critical patient. The queue is implemented using a sorted singly linked list, with the highest-priority patient maintained at the front.

Initially, the queue is:
Patient P1 P2 P3 P4
Priority 2 5 7 9
Front -> [P1,2] -> [P2,5] -> [P3,7] -> [P4,9] -> NULL
The following patients arrive in this order: P5 with priority 1, P6 with priority 6, P7 with priority 10. The system then performs two DEQUEUE operations. Which patients are served?

A. P5, P1
B. P7, P4
C. P1, P2
D. data insufficient to answer the question

47) For an unordered-array implementation of a priority queue, which of the operations requires time proportional to the number of elements in the priority queue?

A. isEmpty()
B. initialize()
C. dequeue()
D. enqueue()

48) Consider a priority queue where higher value means higher priority.

Element	Priority
P	2
Q	5
R	1
S	4

What is the order of dequeuing?

A. P, Q, R, S
B. R, P, S, Q
C. Q, S, P, R
D. S, Q, P, R

49) Suppose smaller priority numbers indicate higher priority. Which job will be processed first?

Job	Priority
J1	4
J2	2
J3	1
J4	3

A. J1
B. J2
C. J3
D. J4

50) A CPU maintains jobs using a priority queue implemented as an unsorted array. A larger number means higher priority. Initially:

Job:	J1	J2	J3	J4	J5
Priority:	20	50	10	40	30

The CPU performs:
1. Insert J6 with priority 60
2. Execute (delete) the highest-priority job
A student claims: "Since J6 has the highest priority and is inserted last, deleting the highest-priority job requires a few operations not proportional to the number of elements in the priority queue." For n jobs, which is the correct conclusion?

A. The student is correct; deletion is always a few operations
B. Insertion is a few operations and deletion is a few operations
C. Insertion is a few operations and deletion requires number of operations proportional to the number of elements in the priority queue
D. Both operations require time proportional to log(n) to base 2 where n is the number of elements in the priority queue.

51) In an unordered-list implementation of a priority queue, where can a newly inserted element generally be placed?

A. Only at the beginning
B. Only at the end
C. At any convenient position, with a few operations
D. Only in sorted order

52) A job scheduler uses a priority queue where a job's priority is dynamically recalculated as (original priority - waiting time), so priority values change while jobs sit in the queue, and lower values are served first. Job X enters with priority 10, Job Y enters with priority 6, and Job Z enters with priority 8. After both X and Z have waited 5 time units in the queue (while Y has waited 0, having just entered), which job is served next?

A. Job Y, since it has the lowest original priority value among all three jobs
B. Job Z, since its effective priority becomes 3, the lowest among the three
C. Job X and Job Z tie exactly, so the scheduler must reject the request
D. Job X, since its effective priority becomes 5, the lowest among the three

53) A student implements a priority queue as an unordered list, inserting every new element at the end of the list without comparing it to existing elements. A classmate argues this design is "broken" because the list looks disorganized. Why is this design still a valid priority queue implementation?

A. Priority queues never need comparisons at any stage of their operation
B. The list automatically becomes sorted the moment a removal operation is called
C. It is not a valid implementation, since a priority queue must always stay sorted
D. Ordering is enforced only during removal, when the highest-priority element is located and extracted

54) An airline boards passengers using a priority queue based on ticket class (Business > Premium > Economy), with FIFO order within the same class. A Business passenger checks in, then two Economy passengers, then another Business passenger. Which passenger boards second?

A. The first Business passenger who checked in
B. The second Business passenger who checked in
C. The first Economy passenger who checked in
D. The second Economy passenger who checked in

55) A hospital emergency system assigns priority as follows: Critical -> Priority 1, Serious -> Priority 2, Normal -> Priority 3. Patients arrive in the order: P1(Normal), P2(Critical), P3(Serious), P4(Critical). If smaller priority number means higher priority and patients with the same priority are served FIFO, what is the service order?

A. P1, P2, P3, P4
B. P2, P4, P3, P1
C. P4, P2, P3, P1
D. P2, P3, P4, P1

56) Consider the following function for an unordered priority queue:

```
node *temp = head;
node *max = head;
while(temp != NULL) {
   if(temp->priority > max->priority) max = temp;
   temp = temp->next;
}
```

What is the purpose of this code?

A. Insert a node
B. Find the highest-priority node
C. Delete the first node
D. Reverse the list

57) Consider the following elements:

Element	Priority
A	3
B	1
C	2

Assuming smaller number means higher priority, which element should be deleted first?

A. A
B. B
C. C
D. All at the same time

58) How does a Priority Queue fundamentally determine which component to remove first during a dequeue operation?

A. It always removes the component that was added first (FIFO).
B. It removes the component with the highest priority first.
C. It always removes the component that was added last (LIFO).
D. It alternates between the highest and lowest values.

59) A hospital ER uses a priority queue where lower numbers = higher priority (1 = critical). Patients arrive with priorities 3, 1, 4, 1, 2 in that order. Ties are broken by arrival order (stable). In what order are they treated?

A. Priority 1 (1st arrival), priority 1 (2nd), priority 2, priority 3, priority 4
B. Priority 4, priority 3, priority 2, priority 1, priority 1
C. Priority 3, priority 1, priority 4, priority 1, priority 2

60) Using an unordered list Priority Queue (lower value = higher priority), the following elements (priority, label) are inserted in this order: (5,A), (2,B), (8,C), (2,D). Ties in priority are broken by insertion order (FIFO). If removeMin() is called twice, which two labels are removed, in order?

A. A, then C
B. B, then D
C. D, then B
D. B, then A

61) If the jobs have the same priority, the one which joined the queue earlier is expected to be dequeued first. Which job should be dequeued in the priority queue implemented using an unordered list with both head and tail?

A. The one nearest to the front with the highest priority
B. The one farthest from the front with the highest priority
C. Any node in the priority queue with the highest priority
D. question is wrong; no two jobs can have the same priority

62) In an ordered array implementation of the priority queue, deque is at the end of the array and not the beginning for reasons of efficiency. If jobs have the same priority, deque the jobs in FIFO order. Where should the job be enqueued?

A. at the end of the array; requires only a few operations
B. at the beginning of the array; may require shifting of jobs
C. to the right of jobs with the same priority
D. to the left of jobs with the same priority

63) If two elements in a priority queue have the same priority, which principle is commonly used?

A. LIFO
B. FIFO
C. Random
D. Reverse ordering

64) When implementing a priority queue using an unordered list, which operation is considered NOT efficient, and why?

A. Enqueue is not efficient because it requires traversing the list to find the absolute end.
B. Dequeue is not efficient because the number of operations in the worst case is proportional to the number of elements in the priority queue.
C. Initialize is not efficient because it must allocate memory for all possible priorities upfront.
D. Check for empty is not efficient because it must scan all elements to verify priority constraints.

65) In a priority queue with higher the value, higher the priority, the following 4 jobs have been serviced in this order: A D B C. Each of them has a distinct priority and the highest priority value is 4 and the least is 1. What would be the priorities of these jobs?

A. A = 4, B = 2, C = 1, D = 3
B. A = 1, B = 3, C = 4, D = 2
C. A = 1, B = 2, C = 3, D = 4
D. cannot decide based on the input provided

66) An unordered-array priority queue stores jobs with ids: [12, 5, 20, 8, 15] and the corresponding priorities are: [not visible/provided in the uploaded source]. If the leftmost highest-priority element is removed, which element is deleted?

A. 12
B. 5
C. 8
D. 20

67) Which representation maintains elements according to priority so that deletion of the highest-priority element can be done efficiently from one end?

A. Unordered list
B. Ordered list
C. Unordered array
D. Simple FIFO queue

68) Which of the following is correct?

A. only enqueue requires number of operations proportional to the number of nodes in the priority queue
B. only dequeue requires number of operations proportional to the number of nodes in the priority queue
C. both require a few operations independent of number of nodes in the priority queue
D. both require number of operations proportional to the number of nodes in the priority queue

69) A priority queue contains the following elements, where larger priority value means higher priority.
Element: A B C D
Priority: 2 4 1 4
The elements are inserted in the order A, B, C, D. If the priority queue follows FIFO for elements having the same priority, what is the order in which the elements are deleted?

A. B, D, A, C
B. D, B, A, C
C. B, A, D, C
D. D, A, B, C

70) In an unordered list implementation of a priority queue, newly element is generally:

A. Inserted according to priority
B. Inserted at any convenient position without maintaining priority order
C. Always inserted at the front
D. Always inserted at the rear

ANSWER KEY - AT THE END

Only the answer key is provided in this final section. Q66 cannot be determined because its corresponding priority values are not visible/provided in the uploaded source. N/A means the answer cannot be determined from the uploaded source.

Q	Ans.	Q	Ans.
1	C	36	C
2	A	37	C
3	B	38	A
4	C	39	C
5	C	40	C
6	A	41	C
7	C	42	A
8	B	43	D
9	D	44	B
10	B	45	A
11	A	46	A
12	C	47	C
13	A	48	C
14	C	49	C
15	B	50	C
16	D	51	C
17	A	52	B
18	D	53	D
19	B	54	B
20	D	55	B
21	B	56	B
22	B	57	B
23	B	58	B
24	B	59	A
25	C	60	B
26	C	61	A
27	C	62	C
28	C	63	B
29	A	64	B
30	C	65	A
31	C	66	N/A
32	A	67	B
33	C	68	B
34	C	69	A
35	B	70	B